

An Argument of Knowledge for a Formal Development

Committing to a machine-checked corpus without disclosing it

Abstract

We hold a large Lean 4 development that passes a strict audit: it builds, it contains no holes, and every theorem in it depends only on the permitted axioms. We want to make that claim publicly checkable without publishing the sources. This note does two things. First, it states precisely what is claimed, in the form of a relation between a public statement and a private witness, and publishes a single digest — a Merkle root over the development — that pins the witness down. Second, it gives an argument of knowledge for that relation and proves the protocol correct *inside the very development it is about*: a three-move public-coin protocol with an explicit knowledge extractor, an explicit simulator, and machine-checked proofs of completeness, of the soundness error under repetition, and of *perfect* honest-verifier zero knowledge — the simulator’s transcript distribution is not close to the honest one, it is equal to it, and the proof of that is a bijection between the two randomness spaces. The binding property of the published digest is reduced, also in Lean, to a SHA-256 collision, using a SHA-256 implementation that lives in the same development. Everything asserted here as a theorem is a theorem in a machine-checked file; everything assumed — collision resistance, the idealisation of the commitment primitive, the random-oracle heuristic behind the non-interactive variant, and one classical NP reduction we did not formalise — is listed explicitly in Section 9.

1 Introduction

The disclosure problem. A formal development is an unusual kind of asset. Its value is that a machine has checked it, and the check is reproducible by anyone who has the sources — but only by someone who has the sources. The usual way to make a claim about such an artifact credible is therefore to hand the artifact over. That is exactly what one may not want to do while the work is unpublished, under review, or commercially sensitive.

The cryptographic literature has an answer to this shape of problem, and it is almost forty years old: a zero-knowledge proof [5] lets a prover convince a verifier of a statement while transferring nothing beyond its truth, and a zero-knowledge *argument of knowledge* strengthens this to “moreover, the prover *has* a witness”, in the strong operational sense that a witness can be computed from a prover who succeeds too often [1]. What has been missing is not the cryptography but the plumbing: what, exactly, is the statement about a Lean development; what pins the witness down; and which parts of the resulting protocol are themselves proved rather than asserted.

What we do. We make the claim about our development into a formal NP statement, publish a commitment to the witness, and give a protocol establishing knowledge of it. We then do to the protocol what we did to the development: we formalise it. The protocol layer is six Lean modules, machine-checked, with no holes, and audited by the same axiom audit that guards the rest of the corpus.

Contributions.

1. **The statement** (Section 2). A precise audit predicate on a development, and the public claim derived from it: a digest, a declaration count and a module count. Formalised as `LeaZk.Artifact.Audited` and `LeaZk.Artifact.Holds`.
2. **The commitment** (Section 3). A Merkle tree over the project’s own verified-shape SHA-256, with a machine-checked *binding* theorem: two different openings of the same position against the same published root yield, constructively, a SHA-256 collision (`LeaZk.Merkle.collision_of_fold_eq`, `LeaZk.Merkle.sha256_collision_of_nodeHash_collision`).
3. **The naive protocol and why it is not enough** (Section 4). Cut-and-choose spot checking, with its exact soundness bound $((m - t)/m)^k$ proved by counting (`LeaZk.Artifact.spotcheck_repetition_le`) — and the observation, also formalised, that every accepting round hands the verifier a piece of the witness.
4. **The zero-knowledge argument** (Sections 5 and 6). A general Σ -protocol framework with completeness, 2-special soundness, the resulting soundness bound, sequential repetition and perfect honest-verifier zero knowledge; and a concrete instantiation, a committed-colouring protocol in the style of Goldreich, Micali and Wigderson [4], with an explicit extractor and an explicit simulator, both proved correct.
5. **The published commitment** (Section 8). The root of our development, the exact algorithm that produced it, and the one command that recomputes it.

What this note deliberately does not contain. No source code of the development, no module names beyond those of the protocol layer itself, no design description, no theorem statements from the corpus. The quantitative claims in Section 2 are aggregates; the digest in Section 8 is what ties them to a specific artifact.

2 The statement

2.1 What an auditor checks

Fix a development $\mathcal{D}$ consisting of m declarations. An auditor’s report on $\mathcal{D}$ is a vector of records, one per declaration, each carrying the outcome of three checks:

1. the declaration elaborated and was accepted by the kernel;
2. its transitive axiom dependencies lie in a permitted set — for us, `propext`, `Classical.choice`, `Quot.sound`, and the two axioms that `native_decide` introduces;
3. no hole (`sorry`, `admit`) occurs in it, equivalently `sorryAx` does not occur in its axiom closure.

Definition 2.1 (Audited). A record is *sound* when all three flags hold; a development is *audited* when every one of its m records is sound. In Lean: `LeaZk.Artifact.Record`, `Record.sound`, `LeaZk.Artifact.Audited`.

This is a decidable predicate on a finite object, which is the whole point: the claim “ $\mathcal{D}$ is audited” is an NP statement whose witness is $\mathcal{D}$ itself, and whose verification is the ordinary work of a proof assistant.

2.2 What is public

The public claim is a triple: a hex digest, a declaration count and a module count (`LeaZk.Artifact.PublicClaim`). The relation the argument of knowledge is for is

$$R = \{ (\text{claim}, \mathcal{D}) : \text{Com}(\mathcal{D}) = \text{claim.root} \wedge \text{claim.declarations} = m \wedge \text{Audit}(\mathcal{D}) \},$$

formalised as `LeaZk.Artifact.Holds`, with the commitment function `Com` a parameter so that the statement does not depend on a choice of hash.

Remark 2.2. Note what the relation does *not* say. It does not say the development is interesting, or correct with respect to any external specification, or that its theorems mean what their names suggest. It says: this many declarations, this digest, every one of them kernel-checked, axiom-clean and hole-free. An argument of knowledge for R is worth exactly that much and no more; the value of the claim is unlocked when the artifact is finally disclosed and the digest is seen to match.

2.3 The aggregates

For context, and without identifying anything, the aggregate measurements of the development at the revision recorded in its own status report: 177 Lean modules and 82 747 lines at that revision; 6 990 declarations, of which 2 953 are theorems and 119 lemmas; 3 072 of 3 072 statements free of `sorry` and `admit`; 25 executable targets and 42 of 42 test suites passing. Adding the protocol layer of this note brings the corpus to 184 Lean files and 26 executables, and the axiom audit that the build itself runs then reports 11 890 project theorems in 146 audited modules, of which 11 886 are kernel-only, 4 rely on `native_decide`, and 0 rely on `sorryAx`. Those numbers are measured by a tool inside the development and re-measured on every push; the argument below is what makes it costly to lie about them.

3 The commitment

3.1 Construction

Leaves are hashed with a domain-separating tag byte `0x00` and an optional salt, internal nodes with the tag byte `0x01`:

$$\text{leaf}(s, d) = H(00 \| s \| d), \quad \text{node}(l, r) = H(01 \| l \| r),$$

where H is SHA-256. A level with an odd number of nodes carries its last node up unchanged. An *opening* of position i is the leaf together with an authentication path: the sibling at each level, tagged with the side it sits on. A verifier folds the path into a root and compares (`LeaZk.Commit.verifyPathBytes`).

3.2 Binding, proved

The property that makes a published root worth publishing is that it cannot be opened two ways. We prove the reduction, for an arbitrary internal hash and with no hardness assumption anywhere in the statement.

Theorem 3.1 (Binding; `LeaZk.Merkle.collission_of_fold_eq`). *Let `node` be any function of two arguments. If two leaves $a \neq b$ have authentication paths of the same shape — the same list of sides, i.e. the same position in the tree — that fold to the same root, then `node` has a collision, and the colliding pair is exhibited by the proof.*

Corollary 3.2 (`LeaZk.Merkle.sha256_collision_of_nodeHash_collision`). *Instantiated at $\text{node}(l, r) = \text{SHA-256}(01||l||r)$ with 32-byte digests, that collision is a pair of distinct byte strings with equal SHA-256 digest.*

So the publisher of a root is bound to one artifact per position, unless they have found a SHA-256 collision. The SHA-256 in question is the one inside the development (transcribed from FIPS 180-4, with its padding, message schedule, block decomposition and hex rendering proved, and its published test vectors decided by the compiler), so the reduction lands on the same code that computes the root.

Theorem 3.3 (`Openings verify`; `LeaZk.Commit.opening_verifies`). *For any list of leaves, any position in it and any hash whatsoever, the authentication path produced by the implementation folds the leaf back to the root. There is no bound on the number of leaves, and the odd levels with their carried nodes are part of the statement. The same claim is re-checked concretely on real SHA-256 digests for trees of one to sixteen leaves (`LeaZk.Commit.openingsVerify_upto_16`), as a cross-check of the proof against the running code.*

Beside that check sits ordinary evidence: the implementation is compared, leaf digest by leaf digest and path step by step, against an independent re-implementation of the same scheme written on top of a standard library SHA-256 (`scripts/compare-zk-with-python.py`). Over trees of 0 to 40 leaves that is 871 agreeing comparisons of roots, openings and derived challenges. Evidence is not proof, and it is reported here as evidence.

3.3 Hiding, assumed

Binding is only half of a commitment. Hiding — that the root reveals nothing about the leaves — is not a theorem about SHA-256 that anyone can prove today; it is modelled by treating H as a random oracle and salting each leaf with fresh randomness. We record this as an assumption (Section 9) rather than dressing it up as a result. For the published root of Section 8 the salt is empty, because there the point is reproducibility on disclosure rather than secrecy of the leaves: the file digests are unguessable in practice but not provably hidden.

4 The naive protocol, and its leak

The obvious protocol is cut-and-choose: the verifier names a position, the prover opens it, and the verifier checks the opening against the root and the audit flags in the record. Its soundness is a counting argument, and we prove it by counting.

Theorem 4.1 (`LeaZk.Artifact.spotcheck_card_accepting`, `LeaZk.Artifact.spotcheck_repetition_le`). *Let t of the m records fail the audit. Then exactly $m - t$ of the m challenges are accepted in one round, and at most $(m - t)^k$ of the m^k challenge vectors are accepted in all of k independent rounds.*

This is sound but useless for our purpose, and the reason is also a (trivial) theorem: `LeaZk.Artifact.spotcheck_reveals` says that the transcript of an accepting round contains the opened record. Convincing a verifier this way means handing over the corpus one declaration at a time. Worse, the bound is weak exactly where it matters: a development with a single bad declaration survives a round with probability $1 - 1/m$, so driving the error below 2^{-40} needs on the order of m times $40 \ln 2$ rounds — and by then the verifier has seen most of the artifact.

Hence the rest of this note.

5 The abstract layer: Σ -protocols

5.1 Definitions

A three-move public-coin protocol (`LeaZk.SigmaProto`) is a triple of functions: a commitment computed from the statement, the witness and the prover’s private randomness; a response computed from those and the verifier’s challenge; and a decision procedure on the transcript. Three properties are formalised.

Definition 5.1 (Completeness; `SigmaProto.Complete`). For every statement x and witness w with $R(x, w)$, every randomness r and every challenge c , the verifier accepts.

Definition 5.2 (2-special soundness; `SigmaProto.SpecialSound`). There is a function $\mathcal{E}$ — an *extractor* — such that from any two accepting transcripts (a, c_1, e_1) and (a, c_2, e_2) that share the commitment a and have $c_1 \neq c_2$, the value $\mathcal{E}(x, a, c_1, e_1, c_2, e_2)$ is a witness for x . This is what “argument of *knowledge*” means here: the extractor is a function in the formalisation, not an existential claim in prose.

Definition 5.3 (Perfect HVZK; `LeaZk.PerfectHVZK`). A simulator $\mathcal{S}$ that sees the statement and the challenge but never a witness, together with, for each statement, witness and challenge, a *bijection* between the prover’s randomness space and the simulator’s under which the two produce literally the same transcript.

The bijection formulation deserves a comment. The textbook statement is that two distributions are equal; a bijection between the underlying finite randomness spaces that matches transcripts pointwise is a strictly stronger, finitely checkable datum, and it implies the distributional statement:

Theorem 5.4 (`LeaZk.PerfectHVZK.transcript_count`, `LeaZk.PerfectHVZK.card_simRand`). *For every transcript t , the number of prover random tapes producing t equals the number of simulator random tapes producing t ; and the two randomness spaces have the same cardinality. Hence the honest and simulated transcript distributions coincide exactly.*

5.2 Soundness and repetition

Theorem 5.5 (`SigmaProto.card_accepting_le_one_of_specialSound`). *If the protocol is 2-specially sound and the statement x has no witness, then any prover strategy whatsoever — described by the first message it sends and the response it is prepared to give to each challenge — is accepted on at most one of the $|C|$ challenges.*

Note the quantification: the cheating prover is an arbitrary pair (commitment, response function), with no computational assumption and no honesty. Two accepting challenges would let the extractor produce the witness that does not exist.

Theorem 5.6 (Repetition; `LeaZk.card_forall_accept_le_pow`). *If in each of k rounds a prover is accepted on at most b challenges, it is accepted in every round on at most b^k of the $|C|^k$ challenge vectors (`LeaZk.card_challenge_vectors`).*

Together: a false statement survives k rounds of a 2-specially sound protocol on at most $|C|^{-k}$ of the coins.

6 The concrete protocol

6.1 From the audit statement to a graph

The audit predicate is in NP, so by the Cook–Levin theorem and the standard chain of reductions it reduces in polynomial time to graph 3-colourability, with a witness-preserving reduction: a

proper colouring of the graph yields an audited development and conversely [3]. We use that reduction as a black box and say so plainly: *it is the one step of this note that we did not formalise* (Section 9). Everything downstream of the graph is machine-checked.

6.2 The committed-colouring protocol

Fix a graph on n vertices with m edges. The prover holds a proper 3-colouring w . One round:

1. the prover draws a uniformly random permutation σ of the three colours and a fresh nonce per vertex, and sends the vector of commitments $(\text{Com}(\sigma(w(z)); \rho_z))_z$;
2. the verifier names an edge i , uniformly at random;
3. the prover opens the two endpoints of i ;
4. the verifier accepts iff the two openings match the commitments sent and the two revealed colours differ (`LeaZk.ThreeColour.verifyStep`).

Formalised as `LeaZk.ThreeColour.proto`, an instance of the abstract shape of Section 5.

Theorem 6.1 (Completeness; `LeaZk.ThreeColour.complete`). *A prover holding a proper colouring is accepted on every edge and for every choice of its randomness.*

6.3 Extraction and soundness

Soundness is proved in the perfectly binding model: a commitment determines the value inside it (`LeaZk.ThreeColour.Binding`). The extractor (`LeaZk.ThreeColour.extract`) reads a colouring straight out of the commitment vector.

Theorem 6.2 (Knowledge extraction; `LeaZk.ThreeColour.extract_proper`). *If a prover can answer every edge challenge acceptably against one and the same first message, then the colouring the extractor reads out of that first message is proper. In particular the prover knows a witness.*

Theorem 6.3 (Soundness; `LeaZk.ThreeColour.card_accepting_le_pred`, `LeaZk.ThreeColour.card_accepting_repetition_le`). *If the graph has no proper colouring, then any prover — again, an arbitrary first message together with an arbitrary response for each edge — is accepted on at most $m - 1$ of the m edges; and over k independent rounds, on at most $(m - 1)^k$ of the m^k challenge vectors.*

So the soundness error after k rounds is at most $(1 - 1/m)^k$, which is below $2^{-\lambda}$ once $k \geq \lambda m \ln 2$. The bound is proved by counting accepting challenges, with no probability space in sight; the statement is about cardinalities of explicit finite sets.

6.4 Perfect zero knowledge

Zero knowledge is proved in the perfectly hiding model: for each value, the commitment is a bijection from nonces to commitments (`LeaZk.ThreeColour.HidingScheme`) — the shape of a Pedersen commitment $g^a h^r$. The simulator (`LeaZk.ThreeColour.simulator`) is given the graph and the challenged edge, invents an ordered pair of distinct colours for its endpoints, commits to that pair and to colour 0 everywhere else, and opens the two endpoints.

The colour-theoretic heart of the matter is small and finite:

Lemma 6.4 (`LeaZk.ThreeColour.permOnPair_bijective`). *For a properly coloured edge with endpoint colours $a \neq b$, the map $\sigma \mapsto (\sigma a, \sigma b)$ from the six permutations of three colours to the six ordered pairs of distinct colours is a bijection.*

Injectivity is the statement that a permutation of a three-element set is determined by its values at two distinct points, decided exhaustively by the kernel (`perm_eq_one_of_fixes_two`); bijectivity then follows from $6 = 6$.

Theorem 6.5 (Perfect HVZK; `LeaZk.ThreeColour.perfectHVZK`). *The protocol has perfect honest-verifier zero knowledge in the sense of Section 5: there is an explicit bijection (`LeaZk.ThreeColour.keyEquiv`) between the prover’s randomness (a permutation and a nonce vector) and the simulator’s (a distinct pair and a nonce vector) under which the transcripts are equal on the nose.*

The bijection is the honest one: the permutation goes to the pair of colours it induces on the challenged edge, and each nonce is re-randomised so that it commits the simulator’s colour to the same commitment the prover sent. On the two opened vertices the re-randomisation is the identity, which is why the openings agree.

Remark 6.6 (Why two models). No commitment scheme is both perfectly binding and perfectly hiding, so the two theorems above are necessarily proved under different idealisations, and a real instantiation gets one of them perfectly and the other computationally. This is the standard state of affairs for this protocol; what is new here is only that both halves are machine-checked, and that the split is visible in the statements rather than buried in a proof sketch.

6.5 Putting the layers together

Corollary 6.7. *Under the assumptions of Section 9, the composite protocol — publish the Merkle root; reduce the audit statement to a graph; run k rounds of the committed-colouring protocol — is a zero-knowledge argument of knowledge that the publisher holds a development of the claimed size whose commitment is the published root and every declaration of which is kernel-checked, axiom-clean and hole-free. Its soundness error is at most $(1 - 1/m)^k$ plus the probability of a SHA-256 collision or of breaking the commitment; its transcripts are simulatable exactly.*

7 Non-interactivity

An interactive protocol needs a live verifier. The Fiat–Shamir transform [2] removes them: the challenges are read off a hash of the public data and the prover’s first messages, so anybody can recompute them and nobody can choose them after the fact. Our implementation derives challenge j from `SHA-256(prefix||j)` reduced modulo the number of edges (`LeaZk.Commit.challenges`), and the command `zkcommit challenges` prints them for a given root.

The transform is sound in the random-oracle model; we do not formalise it, and we flag the standard caveat that Fiat–Shamir applied to a protocol with large soundness error per round requires care in how the rounds are bound together. In this note the non-interactive layer is engineering, not a proved component.

8 The published commitment

The following digest is the Merkle root of the Lean sources of the development, computed by the development itself.

scheme	Merkle tree, SHA-256, tag byte 00 for leaves, 01 for nodes
leaf	$H(00\ path\ 0a\ \text{hex}(H(\text{file contents})))$
node	$H(01\ left\ right)$
odd level	last node carried up unchanged
leaf order	repository-relative paths, ascending
salt	empty
files committed	184
root	4d71ac822c77cfe74d3dfc438f3f7723d006f2c2637c7860540b15597eaae537

How to check it. Anyone holding the artifact recomputes the root with one command, `lake exe zkcommit root RequestProject`, and compares. Anyone *not* holding the artifact can ask for openings: `lake exe zkcommit open` emits the leaf and its authentication path for a position, and Theorem 3.1 says those openings cannot be forged without a SHA-256 collision. Opening reveals the corresponding file digest and path, which is precisely what the zero-knowledge protocol of Section 6 avoids.

9 What is proved and what is assumed

9.1 Proved, machine-checked, hole-free

result	Lean name, in RequestProject/Zk/
completeness (abstract)	Sigma: SigmaProto.Complete (definition)
knowledge extraction (abstract)	Sigma: SigmaProto.SpecialSound (definition)
soundness, one round	Sigma: card_accepting_le_one_of_specialSound
honest prover always accepted	Sigma: card_accepting_honest
sequential repetition	Sigma: card_forall_accept_le_pow
simulated = honest distribution	Sigma: PerfectHVZK.transcript_count
recolouring is uniform	ThreeColour: permOnPair_bijective
completeness (concrete)	ThreeColour: complete
extraction (concrete)	ThreeColour: extract_proper
soundness (concrete, k rounds)	ThreeColour: card_accepting_repetition_le
perfect HVZK (concrete)	ThreeColour: perfectHVZK
Merkle binding	Merkle: collision_of_fold_eq
binding at SHA-256	Merkle: sha256_collision_of_nodeHash_collision
spot-check counting	Artifact: spotcheck_repetition_le
openings verify (any tree)	CommitProofs: opening_verifies
the same, on real digests	CommitProofs: openingsVerify_upto_16

All of these are audited by the same mechanism as the rest of the development: every module is imported by an audit module that walks the environment and fails the build if any theorem’s transitive axiom closure leaves the permitted set.

9.2 Assumed

Assumption 9.1 (Collision resistance). SHA-256 is collision resistant. Theorem 3.1 is a reduction, not a hardness proof: it converts a double opening into a collision.

Assumption 9.2 (Commitment idealisation). Soundness is proved for a perfectly binding commitment and zero knowledge for a perfectly hiding one. A concrete instantiation satisfies one perfectly and the other under a computational assumption (discrete logarithm, for Pedersen).

Assumption 9.3 (Random oracle, for hiding and for Fiat–Shamir). Hiding of hash-based leaf commitments, and soundness of the non-interactive variant, are argued in the random-oracle model and are not formalised.

Assumption 9.4 (The NP reduction). The witness-preserving reduction from the audit predicate to graph 3-colourability is classical and is used as a black box; it is not formalised. A fully formal end-to-end chain would need Cook–Levin in Lean.

Assumption 9.5 (Verifier honesty). Zero knowledge is proved in the honest-verifier sense. The standard upgrade to malicious verifiers (via coin-tossing, or via the Fiat–Shamir transform in the non-interactive setting) is not formalised.

Assumption 9.6 (The audit predicate is the right one). That kernel-checked, axiom-clean and hole-free is what one should want of a formal development is a judgement, not a theorem. The protocol is agnostic: any decidable per-declaration predicate slots into the same relation.

10 Related work

Zero-knowledge proofs are due to Goldwasser, Micali and Rackoff [5]; the fact that every NP statement has one, given commitments, is due to Goldreich, Micali and Wigderson [4], and the protocol of Section 6 is theirs. Proofs of knowledge in the extractor sense are standard, following Bellare and Goldreich [1]; Σ -protocols in the shape used here go back to Schnorr’s identification scheme [8]. The removal of interaction by hashing the transcript is Fiat and Shamir [2]. Merkle trees are Merkle’s [6]. Hiding commitments with the bijective structure we assume are Pedersen’s [7]. The NP-completeness of graph 3-colourability, and the reductions we invoke as a black box, are in Garey and Johnson [3].

11 Limitations

The chain from “the development is audited” to “the verifier is convinced” has one unformalised link (the NP reduction) and three modelling assumptions (collision resistance, commitment idealisation, random oracle). The protocol as formalised is honest-verifier and interactive. The soundness error per round of the committed-colouring protocol is $1 - 1/m$, so the round count is linear in the size of the graph: this is the classical protocol, not a succinct one, and a practical deployment would want a modern argument system instead — at the cost of a far larger trusted surface, which is exactly what this note tries to keep small.

What the note does establish is that the small surface can be made machine-checked end to end on the parts that are combinatorial: extraction, soundness under repetition, and perfect simulation are theorems in a file that a kernel has read, in the same corpus, under the same audit, as the work they are about.

References

- [1] M. Bellare and O. Goldreich. On defining proofs of knowledge. *Advances in Cryptology — CRYPTO ’92*, LNCS 740, 1993.
- [2] A. Fiat and A. Shamir. How to prove yourself: practical solutions to identification and signature problems. *Advances in Cryptology — CRYPTO ’86*, LNCS 263, 1987.
- [3] M. R. Garey and D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. Freeman, 1979.
- [4] O. Goldreich, S. Micali and A. Wigderson. Proofs that yield nothing but their validity, or all languages in NP have zero-knowledge proof systems. *Journal of the ACM* 38(3), 1991.
- [5] S. Goldwasser, S. Micali and C. Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing* 18(1), 1989.

- [6] R. C. Merkle. A digital signature based on a conventional encryption function. *Advances in Cryptology — CRYPTO '87*, LNCS 293, 1988.
- [7] T. P. Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. *Advances in Cryptology — CRYPTO '91*, LNCS 576, 1992.
- [8] C. P. Schnorr. Efficient signature generation by smart cards. *Journal of Cryptology* 4(3), 1991.

A The protocol, as formalised

For readers who want the shape of the definitions without the sources, the abstract protocol and its three properties are, in ASCII transliteration of the Lean:

```
structure SigmaProto (Stmt Wit Rand Com Chal Resp : Type) where
  commit  : Stmt -> Wit -> Rand -> Com
  respond : Stmt -> Wit -> Rand -> Chal -> Resp
  verify  : Stmt -> Com -> Chal -> Resp -> Bool

Complete S R      : forall x w, R x w -> forall r c,
                    S.verify x (S.commit x w r) c (S.respond x w r c) = true

SpecialSound S R ext :
  forall x a c1 e1 c2 e2, c1 <> c2 ->
    S.verify x a c1 e1 = true -> S.verify x a c2 e2 = true ->
    R x (ext x a c1 e1 c2 e2)

structure PerfectHVZK S R where
  SimRand : Type
  sim      : Stmt -> Chal -> SimRand -> Com * Resp
  key      : forall x w, R x w -> Chal -> (Rand ~ SimRand)
  sim_eq   : forall x w (h : R x w) r c,
            sim x c (key x w h c r) = (S.commit x w r, S.respond x w r c)
```

The soundness theorem quantifies over an arbitrary cheating strategy, given as a first message together with a response for each challenge:

```
structure Cheater (Com Chal Resp : Type) where
  com  : Com
  resp : Chal -> Resp

card_accepting_le_one_of_specialSound :
  SpecialSound S R ext -> (forall w, not (R x w)) ->
  forall P : Cheater Com Chal Resp,
    (univ.filter (fun c => S.verify x P.com c (P.resp c) = true)).card <= 1
```

and repetition multiplies the counts:

```
card_forall_accept_le_pow (acc : Fin k -> Chal -> Bool) (b : Nat) :
  (forall j, (univ.filter (fun c => acc j c = true)).card <= b) ->
  (univ.filter (fun v : Fin k -> Chal => forall j, acc j (v j) = true)).card
  <= b ^ k
```